

# Kalman Filtering I: The Discrete Kalman Filter from Scratch

Optimal estimation of a noisy state, coded predict-then-update.

A Luenberger observer (see `State-Space/StateObservers.m`) places the estimation-error poles by hand. The **Kalman filter** instead chooses the observer gain **optimally** at every step, given how noisy the process and the measurements are. In this tutorial you will:

- code the predict/update recursion by hand,
- track a noisy constant-velocity target from position measurements only, and
- watch the error covariance – and hence the gain – settle.

Uses only core MATLAB. Run with `publish('Intro.m')`, or step through with **Ctrl+Enter**.

## Contents

- Model: a constant-velocity target
- Generate a truth trajectory and noisy measurements
- The Kalman recursion, by hand
- Estimate vs. measurement vs. truth
- Notice: the gain and covariance settle
- What changed: filtered error vs. raw measurement error
- Try it yourself

## Model: a constant-velocity target

State  $x = [p, \dot{p}]^T$ . Between samples the target coasts at constant velocity, nudged by a small random acceleration (**process noise**). We measure position only, corrupted by **sensor noise**.

$$x_{k+1} = Ax_k + w_k, \quad y_k = Cx_k + v_k.$$

```
dt = 0.1;
A = [1 dt; 0 1];
C = [1 0];
G = [dt^2/2; dt]; % how a random acceleration enters the state
q = 0.2;          % process-noise variance (random accel)
R = 4;            % measurement-noise variance (sensor)
Q = G*q*G';       % process-noise covariance
```

## Generate a truth trajectory and noisy measurements

In a real experiment only `Ymeas` would be available; we keep the truth `X` to grade the estimate later.

```
rng(0);
N = 100;
x = [0; 1]; % start at position 0, moving at 1 unit/s
X = zeros(2,N); Ymeas = zeros(1,N);
for k = 1:N
    x = A*x + G*sqrt(q)*randn; % coast + random accel kick
    X(:,k) = x;
    Ymeas(k) = C*x + sqrt(R)*randn; % noisy position measurement
end
```

## The Kalman recursion, by hand

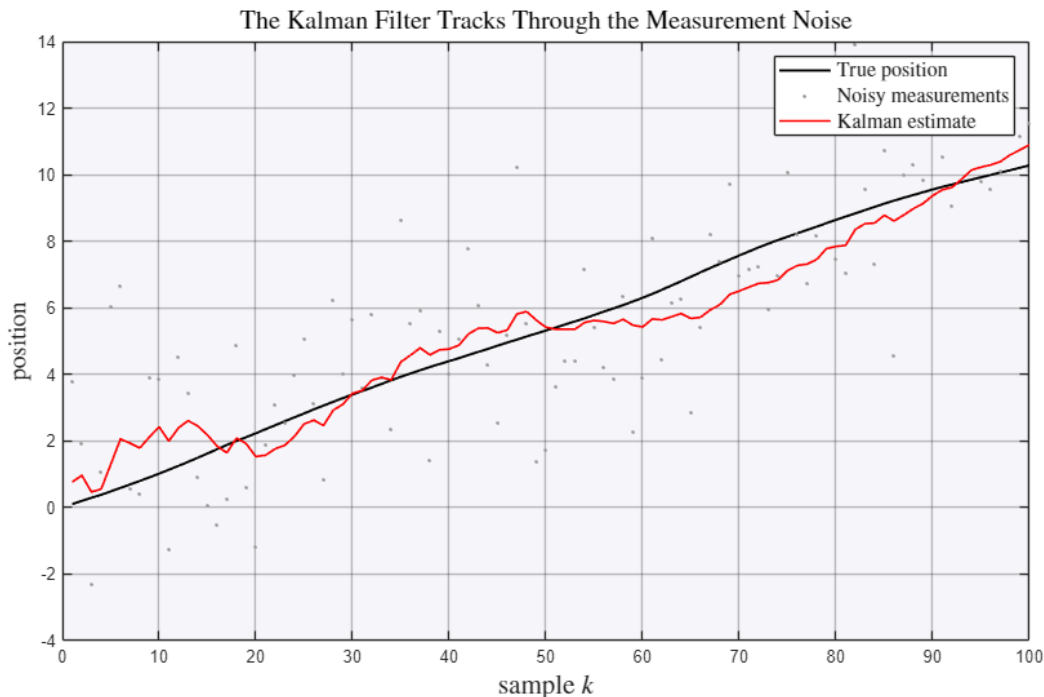
Two steps per sample:

- **Predict:**  $\hat{x} \leftarrow A\hat{x}$ ,  $P \leftarrow APA^T + Q$
- **Update:**  $K = PC^T(CPC^T + R)^{-1}$ ,  $\hat{x} \leftarrow \hat{x} + K(y - C\hat{x})$ ,  $P \leftarrow (I - KC)P$

```
xhat = [0; 0];
P = eye(2);
Xhat = zeros(2,N); Khist = zeros(2,N); Ptrace = zeros(1,N);
for k = 1:N
    % predict
    xhat = A*xhat;
    P = A*P*A' + Q;
    % update
    K = P*C' / (C*P*C' + R);
    xhat = xhat + K*(Ymeas(k) - C*xhat);
    P = (eye(2) - K*C)*P;
    Xhat(:,k) = xhat; Khist(:,k) = K; Ptrace(k) = trace(P);
end
```

## Estimate vs. measurement vs. truth

```
figure
plot(1:N, X(1,:), 'k', 'LineWidth', 1.4)
hold on
plot(1:N, Ymeas, '.', 'Color', [.6 .6 .6])
plot(1:N, Xhat(1,:), 'r', 'LineWidth', 1.2)
hold off
grid on
legend('True position', 'Noisy measurements', 'Kalman estimate', 'Interpreter', 'latex', 'FontSize', 12)
title('The Kalman Filter Tracks Through the Measurement Noise', 'Interpreter', 'latex', 'FontSize', 15)
ylabel('position', 'Interpreter', 'latex', 'FontSize', 14)
xlabel('sample $k$', 'Interpreter', 'latex', 'FontSize', 16)
```



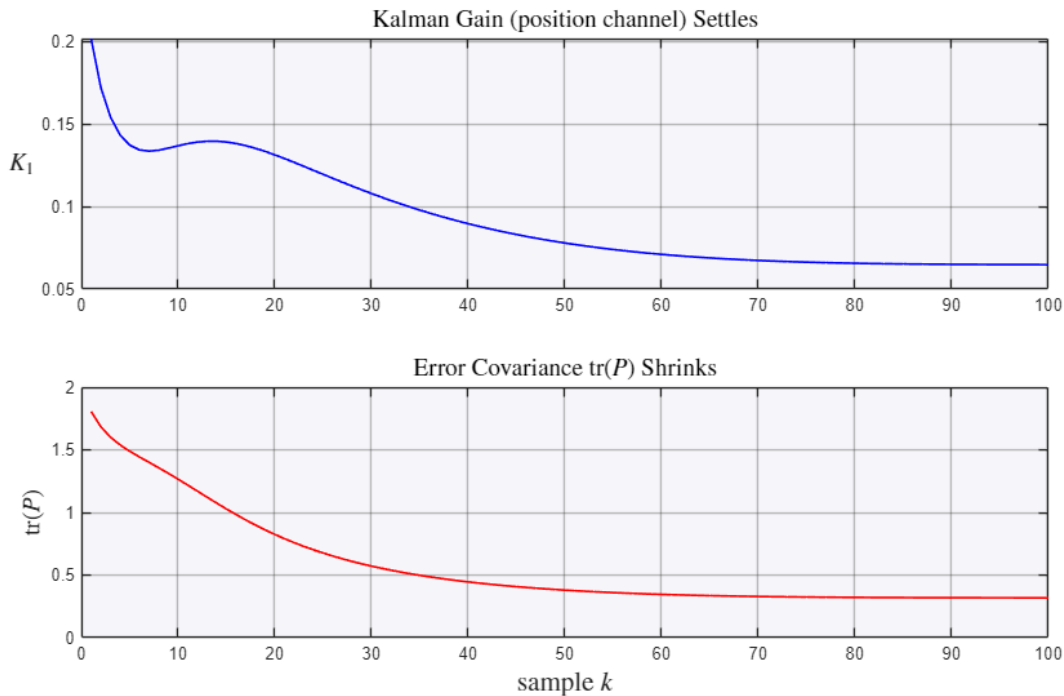
## Notice: the gain and covariance settle

**Notice that** the filter is not a fixed low-pass. The gain  $K$  starts large (trust the measurements while the estimate is uncertain) and settles to a constant as the error covariance  $P$  converges. That steady value is what `SteadyStateKalman.m` computes in one line.

```

figure
subplot(2,1,1)
plot(1:N, Khist(1,:), 'b', 'LineWidth', 1.2)
grid on
title('Kalman Gain (position channel) Settles', 'Interpreter', 'latex', 'FontSize', 14)
ylabel('$K_1$', 'Interpreter', 'latex', 'FontSize', 14); set(get(gca, 'YLabel'), 'Rotation', 0)
subplot(2,1,2)
plot(1:N, Ptrace, 'r', 'LineWidth', 1.2)
grid on
title('Error Covariance $\mathrm{tr}(P)$ Shrinks', 'Interpreter', 'latex', 'FontSize', 14)
ylabel('$\mathrm{tr}(P)$', 'Interpreter', 'latex', 'FontSize', 14)
xlabel('sample $k$', 'Interpreter', 'latex', 'FontSize', 16)

```



## What changed: filtered error vs. raw measurement error

Fusing the model with the data cuts the RMS position error well below the raw sensor noise.

```

rms_meas = sqrt(mean((Ymeas - X(1,:)).^2));
rms_kf    = sqrt(mean((Xhat(1,:) - X(1,:)).^2));
fprintf('RMS position error -- raw measurement: %.3f, Kalman estimate: %.3f\n', ...
        rms_meas, rms_kf)

```

```
RMS position error -- raw measurement: 2.178, Kalman estimate: 0.709
```

## Try it yourself

- Raise the measurement noise  $R$  and notice the steady gain drop – the filter leans on the model and distrusts the noisy sensor.
- Raise the process noise  $q$  and watch the opposite: the gain climbs as the filter trusts fresh measurements over a less certain model.